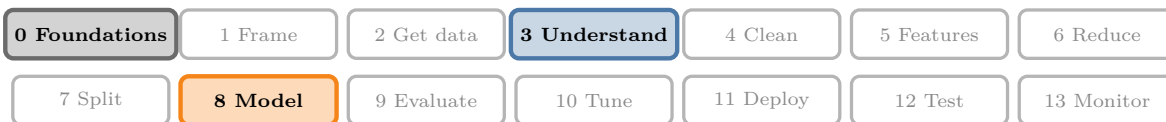


Naive Bayes with Numerical Data: Gaussian Naive Bayes

Contents

1. Overview	2
2. The problem	2
3. The assumption: normal distributions	4
4. The numbers	5
5. Adding logs instead of multiplying	7
6. Reading the result	7
6.1 Which feature decides	7
6.2 Where the prediction flips	8
7. When the data is not normal	9
8. Summary	10
9. Sources	10
10. Key terms	10

Where this fits: Pipeline map steps 0 (Foundations), 3 (Understand data), 8 (Model). See the [Course map](#).



- **Builds on:** Classification problems ([Note 3](#)); Independent and mutually exclusive events ([Note 84](#)); Bayes' theorem ([Note 86](#)).
- **Leads to:** Cumulative distribution function (CDF) ([Note 241](#)); Density estimation ([Note 243](#)); Z-score outlier method ([Note 251](#)); Standard normal and the z-table ([Note 251](#)); Q-Q plot ([Note 260](#)); Central limit theorem ([Note 271](#)).
- **Compare with:** Student's t-distribution ([Note 282](#)); Likelihood ([Note 630](#)).

1. Overview

Key point: Numerical features rarely repeat exactly, so counting fails. Gaussian Naive Bayes assumes each feature follows a normal distribution within each class, and uses the normal curve's height (density) in place of a counted probability.

So far every **feature** (G-772; input variable, one column of the data table) was categorical, so probabilities came from counting. This Note handles **numerical** features such as height or weight, and introduces **Gaussian Naive Bayes** (G-830), the most common variant for them.

This Note covers:

- why counting fails for numbers (section 2);
- the assumption: one normal curve per class and feature (section 3);
- the scores for a new person, read off the curves (section 4);
- adding logs instead of multiplying small numbers (section 5);
- reading the result: which feature decides, and where the prediction flips (section 6);
- other variants for features that are not normal (section 7).

2. The problem

Key point: No man in the data is exactly 185 cm tall, so $P(\text{height} = 185 \mid \text{male})$ by counting is 0.

The data has 8 people. Each person is one **observation** (G-1374; one record, a row of the table). The two features are height (cm) and weight (pounds); the **target** (G-1949; the output we predict) is gender:

Height (cm)	Weight (lb)	Gender
182.9	180	male
180.4	190	male

Height (cm)	Weight (lb)	Gender
170.1	170	male
180.4	165	male
152.4	100	female
167.6	150	female
165.2	130	female
175.3	150	female

A new person is 185 cm tall and weighs 170 pounds. Male or female?

The Naive Bayes recipe (the maths Note) needs

$$P(\text{male}) \times P(\text{height} = 185 \mid \text{male}) \times P(\text{weight} = 170 \mid \text{male})$$

and the same for female. The **prior** (G-1565), the share of each class in the data, is easy: 4 of 8 are male, so $P(\text{male}) = 1/2$. But no man in the data is exactly 185 cm, so counting gives $P(\text{height} = 185 \mid \text{male}) = 0$, and the whole score is 0. With a continuous measurement, almost every new value is one never seen before, so counting cannot work.

$P(\text{height} = x \mid \text{male})$ by counting the 4 men

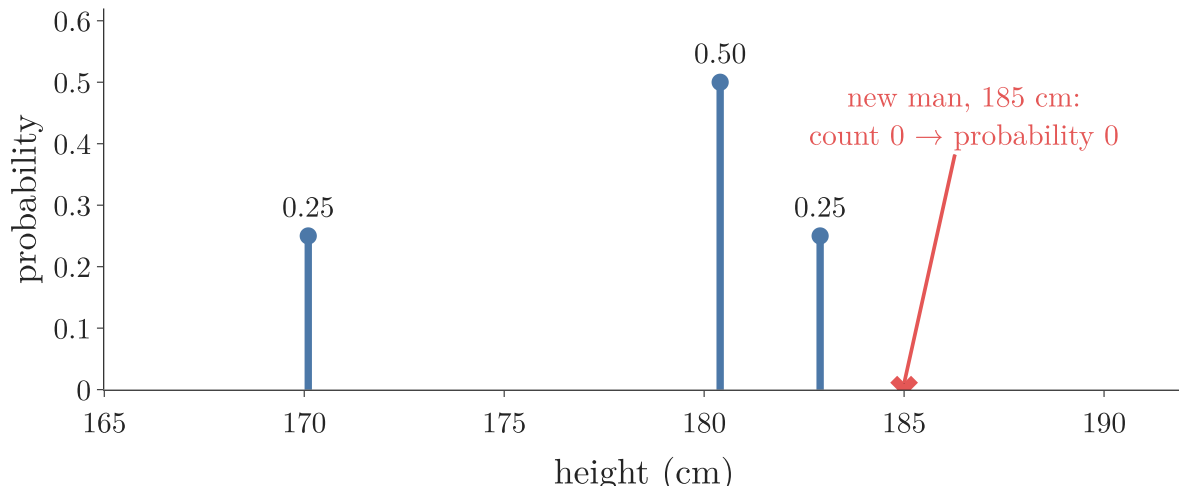


Figure 1: $P(\text{height} \mid \text{male})$ by counting: a spike at each of the 4 male heights, and nothing at 185 cm

In Figure 1, watch the gap at 185 cm: counting puts all its probability on the exact heights already seen, so a man just 2 cm taller than the tallest one gets probability 0.

3. The assumption: normal distributions

Key point: Assume each feature is normally distributed within each class. Estimate the mean and standard deviation, then read off the curve's height at the new value.

Gaussian Naive Bayes assumes that, within each class, each numerical feature follows a **normal** (Gaussian) **distribution** (G-1343), the bell curve of the earlier statistics Notes.

For each class and each feature:

1. compute the **mean** (G-1203) μ and **standard deviation** (G-1871) σ of that feature over that class's observations;
2. for a new value x , compute the height of that class's normal curve at x , the **probability density** (G-1569):

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

3. use $f(x)$ in place of $P(x \mid \text{class})$ in the Naive Bayes product. In this role the curve's height is the **likelihood** (G-1086) of the value under that class.

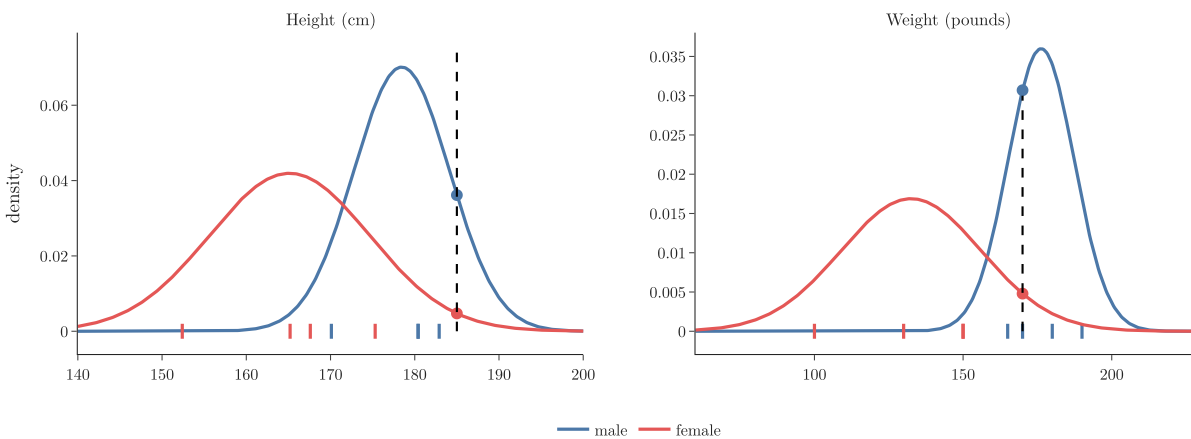


Figure 2: A normal curve per class for each feature; the dots mark the new person's values

Figure 2 shows the four fitted curves. The new person's height (185 cm) sits on the right flank of the male curve but far out in the tail of the female curve; the same holds for the weight (170 pounds).

4. The numbers

Key point: Male score 5.5×10^{-4} against female 1.1×10^{-5} : predict male, with about 98% probability.

Class	Feature	Mean	Std. deviation	Density at the new value
male	height	178.45	5.69	0.03615
male	weight	176.25	11.09	0.03070
female	height	165.12	9.51	0.00473
female	weight	132.50	23.63	0.00479

$$\text{male: } 0.5 \times 0.03615 \times 0.03070 = 5.5 \times 10^{-4}$$

$$\text{female: } 0.5 \times 0.00473 \times 0.00479 = 1.1 \times 10^{-5}$$

Figure 3 shows where the four densities come from. A dashed line marks the new person's value on each chart; the height of each curve at that line is the density.

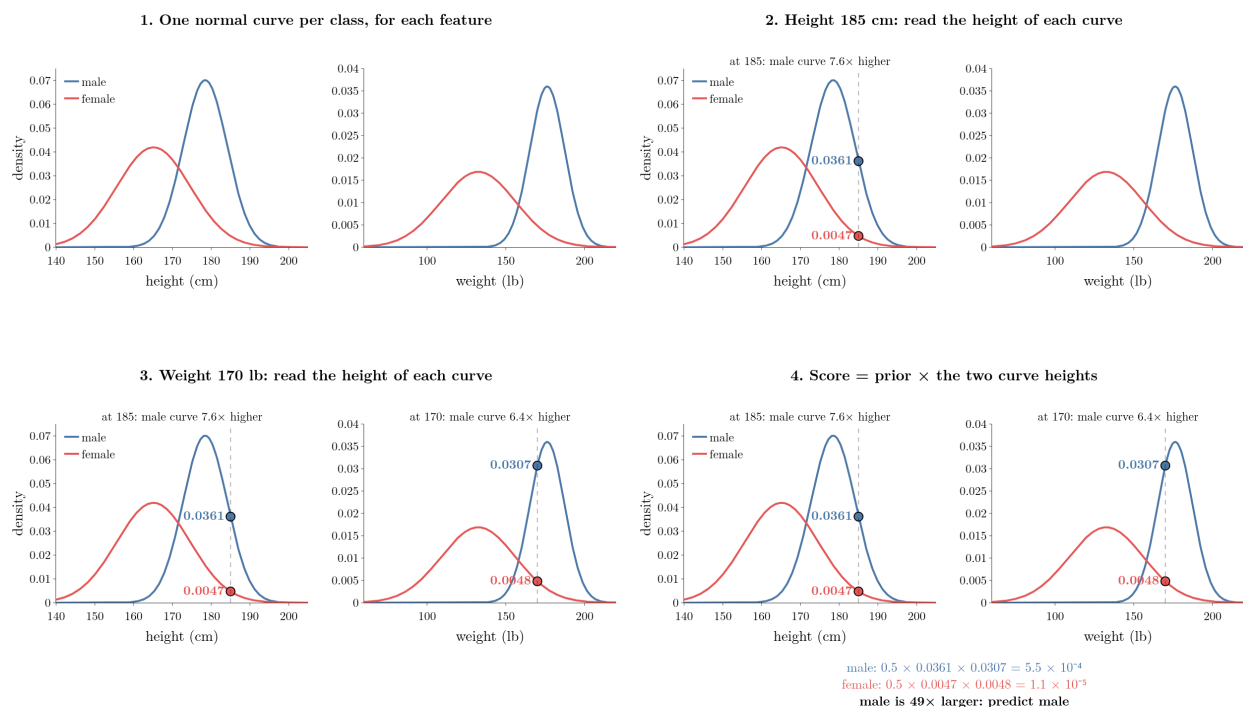


Figure 3: Reading the likelihoods off the curves. At 185 cm the male curve is 0.0361 high and the female curve 0.0047. At 170 lb the heights are 0.0307 and 0.0048. Prior $\times$ both heights gives the two scores. Idea after StatQuest, “Gaussian Naive Bayes, Clearly Explained!!!”; the data are ours.

The male score is about 49 times larger, so the prediction is **male**. As probabilities: 98% male, 2% female.

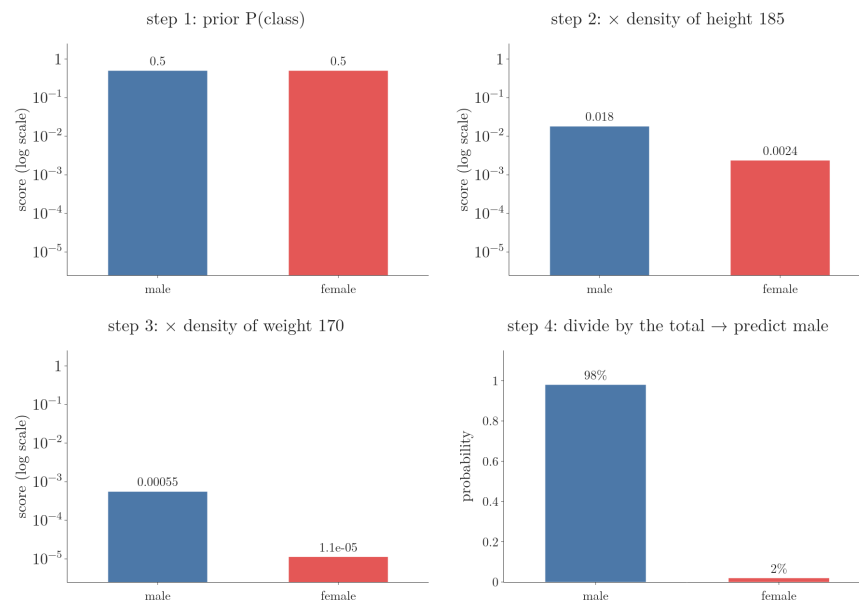


Figure 4: The two scores built one factor at a time (log scale), then divided by their total

In Figure 4, watch the gap open: both classes start at 0.5, the height density pulls the female score about 8 times lower, and the weight density widens the gap to about 49 times.

Extra: A density is not a probability: for a continuous variable, the probability of exactly 185.000... cm is 0, and densities can even be larger than 1. But the density measures how likely values **near** 185 are, and the same small interval would multiply every class's density equally. So comparing densities across classes gives the same decision as comparing probabilities.

Python: scikit-learn's GaussianNB.

```
from sklearn.naive_bayes import GaussianNB
```

```
nb = GaussianNB().fit(df[["height_cm", "weight_lb"]], df["gender"])
nb.theta_           # class means
nb.var_             # class variances
nb.predict_proba(new_person)  # about 99% male
```

GaussianNB estimates the variance by dividing by n rather than $n - 1$ (and adds a tiny amount, `var_smoothing`, for numerical safety; scikit-learn docs, `GaussianNB`), so its numbers differ slightly from the table: 99.2% male instead of 98.0%. The prediction is the same.

5. Adding logs instead of multiplying

Key point: Many small factors multiplied together can become too small for the computer to store. Taking the log of each factor and adding gives the same winner, with ordinary-sized numbers.

The two scores are already small: 5.5×10^{-4} and 1.1×10^{-5} , with only two features. Every extra feature multiplies in another small density. With hundreds of features the product falls below the smallest number the computer can store and becomes 0 for every class. This failure is **underflow** (G-2036), met before in the [log loss Note](#).

The fix is to take the natural log of the score. The log of a product is the sum of the logs, so the multiplication becomes an addition:

$$\ln(\text{score}) = \ln P(\text{class}) + \ln f(\text{height}) + \ln f(\text{weight})$$

On the new person:

	ln prior	ln density of height	ln density of weight	Sum
male	$\ln 0.5 = -0.69$	$\ln 0.03615 = -3.32$	$\ln 0.03070 = -3.48$	-7.50
female	$\ln 0.5 = -0.69$	$\ln 0.00473 = -5.35$	$\ln 0.00479 = -5.34$	-11.39

The log is an increasing function, so the class with the larger score also has the larger log score. -7.50 is larger than -11.39, so the prediction is male, as before. scikit-learn computes Naive Bayes with log scores for this reason (its **GaussianNB** works with a joint log-likelihood).

6. Reading the result

Key point: Each feature's say is the ratio of its two curve heights. Here height favours male 7.6 times and weight 6.4 times. With the weight fixed at 170 lb, the prediction flips from female to male at about 166 cm.

6.1 Which feature decides

Key point: The feature whose two curve heights differ most has the largest say.

Both priors are 0.5, so the 49-fold gap between the scores comes only from the densities. Figure 3 gives each feature's share:

- height: $0.03615/0.00473 = 7.6$ times in favour of male;
- weight: $0.03070/0.00479 = 6.4$ times in favour of male;
- together: $7.6 \times 6.4 \approx 49$.

Here the two features have a similar say. When one feature's ratio is far larger than all the others, that feature alone decides the class, and the others may not be needed. **Cross-validation** (G-510, [Note 29](#)) can check which features help.

6.2 Where the prediction flips

Key point: The set of inputs where the two classes are equally likely is the decision boundary.

Figure 5 keeps the weight at 170 lb and slides the height from 150 to 190 cm. On the left, the dashed line moves across the two height curves. On the right, the probability of male is traced out.

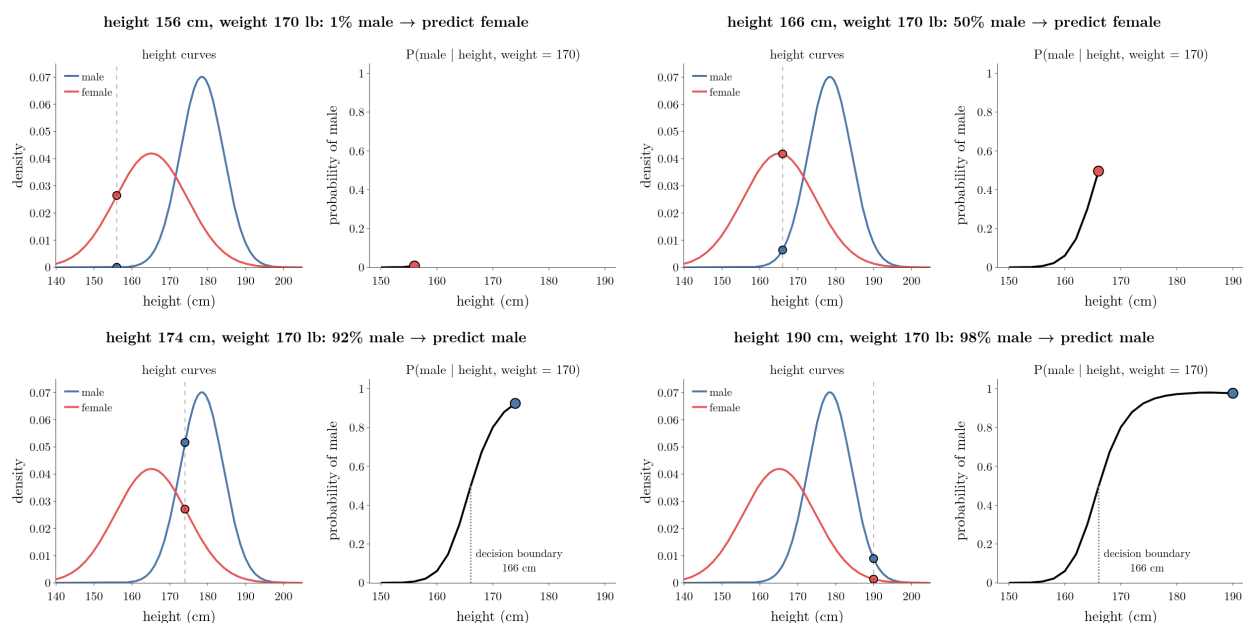


Figure 5: Sliding the height with the weight fixed at 170 lb. Left: the two height curves and the current height. Right: $P(\text{male})$ for each height. The curve crosses 50% at about 166 cm, the decision boundary

Watch the right-hand chart: for short heights the female curve is much higher and $P(\text{male})$ is near 0. Near 166 cm the two scores are equal and $P(\text{male})$ is 50%. Above that the male score wins. The point where the prediction switches is the **decision boundary** (G-555). It sits below the crossing point of the two height curves (about 172 cm) because the weight of 170 lb already favours male 6.4 times.

7. When the data is not normal

Key point: The normal assumption can be wrong. Other variants assume other distributions: multinomial for counts, Bernoulli for yes/no, categorical for categories.

The normal assumption is a choice, and it can be poor, for example for a skewed feature like income. Naive Bayes can use any distribution that suits the data, giving different variants:

Variant	Assumed distribution	Typical features	scikit-learn
Gaussian	normal	continuous measurements	<code>GaussianNB</code>
Multinomial	multinomial	counts, such as word counts in a document	<code>MultinomialNB</code>
Bernoulli	Bernoulli (yes/no)	binary features, such as “word present or not”	<code>BernoulliNB</code>
Categorical	categorical	categories, as in the Play Tennis Note	<code>CategoricalNB</code>

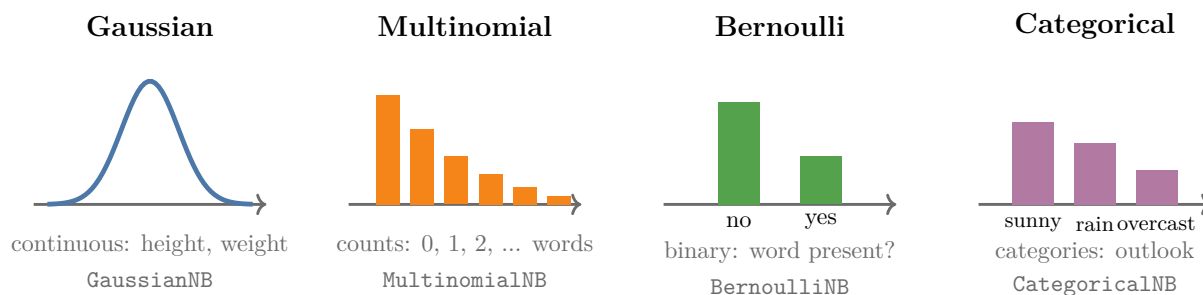


Figure 6: The shape each variant assumes for one feature within one class

In Figure 6, match the shape to the feature: a smooth bell for measurements, bars over 0, 1, 2, ... for counts, two bars for yes/no, one bar per category.

Each variant suits one kind of data (scikit-learn user guide §1.9), so we look at each feature’s distribution and pick the variant whose assumption fits it. A strongly skewed numerical feature can also be transformed first (the power transformer Note) so that it looks more normal.

8. Summary

- Numerical values rarely repeat, so $P(x \mid \text{class})$ cannot be counted.
- Gaussian Naive Bayes fits a normal curve (mean, standard deviation) per class and feature, and uses the density at x .
- New person 185 cm, 170 lb: male score 5.5×10^{-4} , female 1.1×10^{-5} : male (98%).
- With many features, add the logs of the factors instead of multiplying them, to avoid underflow.
- A feature's say is the ratio of its two curve heights; the decision boundary is where the scores are equal.
- Other variants (multinomial, Bernoulli, categorical) suit other kinds of features.

9. Sources

Built from

- CampusX, “Naive Bayes Part 9 | Handling Numerical Data”, YouTube, <https://www.youtube.com/watch?v=TCgK2nBJx9o>
- StatQuest with Josh Starmer, “Gaussian Naive Bayes, Clearly Explained!!!”, YouTube, <https://www.youtube.com/watch?v=H3EjCKtIVog> (the likelihood as the curve's height, adding logs to avoid underflow, one feature having the larger say; Figure 3 is redrawn from this idea with our own data)

Other references

- **scikit-learn docs:** `sklearn.naive_bayes.GaussianNB` (`var_smoothing`); user guide Section 1.9, “Naive Bayes”, scikit-learn 1.9.

10. Key terms

Term	Meaning
Gaussian Naive Bayes (G-830)	Naive Bayes that models each numerical feature as normally distributed within each class
Probability density (G-1569)	The height of a continuous distribution's curve; compares how likely nearby values are
GaussianNB	scikit-learn's Gaussian Naive Bayes
Underflow (G-2036)	A number too close to 0 for the computer to store, which then becomes 0 or loses precision
Decision boundary (G-555)	The set of inputs where the model's prediction switches from one class to the other
MultinomialNB (G-1277)	Naive Bayes for count data, such as word counts
BernoulliNB (G-277)	Naive Bayes for binary (yes/no) features